

UF8

Aplicaciones web Híbridas

Desarrollo web en entorno servidor

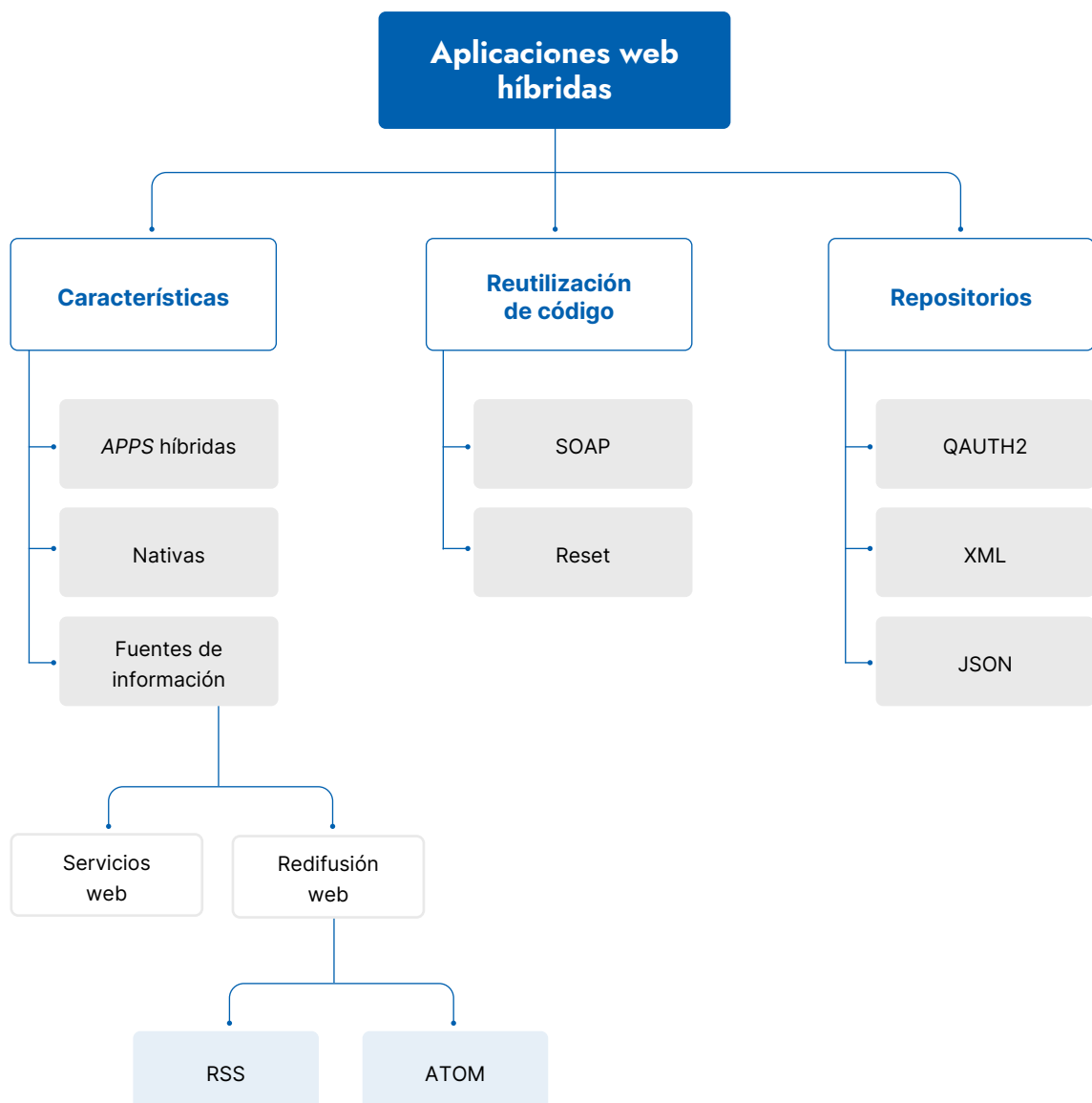
ÍNDICE

Mapa conceptual	04
1. Aplicaciones web híbridadas	05
1.1. Concepto de aplicación web híbrida	05
1.2. Arquitectura de los <i>mashups</i>	06
1.3. Tipos de <i>mashups</i>	08
A. Tipos de <i>mashups</i> atendiendo al contenido	08
B. Tipos de <i>mashup</i> atendiendo a su localización	09
1.4. Ventajas e inconvenientes del uso de <i>mashups</i>	11
2. Ventajas y mecanismos para la reutilización de código	12
2.1. ¿Reutilización de código?	12
2.2. Arquitectura de una aplicación web híbrida	13
2.3. Comunicación en la arquitectura <i>mashup</i>	15
2.4. División por categorías de los <i>mashups</i>	16
A. <i>Mashups</i> de mapas	16
B. <i>Mashups</i> de fotos y vídeo	16
C. <i>Mashups</i> de compras	16
D. <i>Mashups</i> de noticias	16
2.5. Futuro de los <i>mashups</i>	17
3. Repositorios	18
3.1. OAuth2	18
3.2. JWT	20
3.3. JSON y XML	20
3.4. Elaboración de documentos con formato RSS	22

4. Creación de aplicaciones web híbridas25

4.1. Herramientas utilizadas para crear <i>mashups</i> o aplicaciones híbridas.....	25
A. Google Maps	26
B. Amazon Web Services	27
C. Flickr.....	27
D. YouTube	27
E. eBay.....	28
F. Yahoo!	28
4.2. Librerías PHP de acceso a servicios web para la creación de aplicaciones híbridas.....	28

Mapa conceptual



01 Aplicaciones web híbridas

1.1. Concepto de aplicación web híbrida

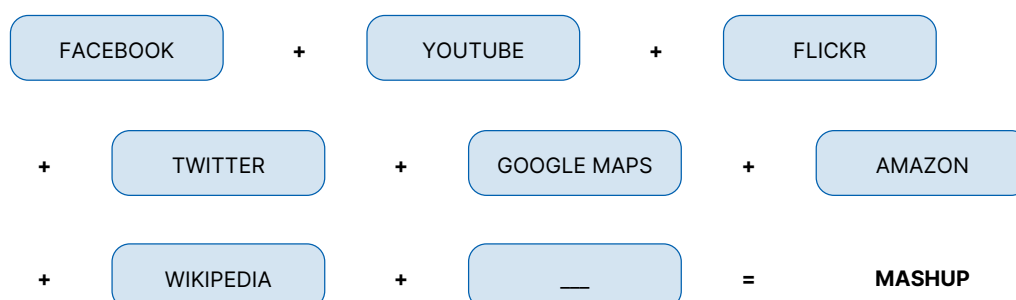
Una aplicación web híbrida, también conocida por su nombre en inglés (*mashup*), se caracteriza por combinar datos o funcionalidades procedentes de diversos orígenes para formar un nuevo tipo de aplicación o servicio.

Un **mashup** o aplicación web híbrida es una aplicación que combina servicios de varias páginas web en una sola para ofrecer un nuevo servicio. Es habitual que la integración de datos se obtenga de API abiertas. La libertad de integrar todas estas fuentes es una oportunidad para desarrollar en muy poco tiempo nuevos servicios, basados en funcionalidades ya existentes.

En esencia, la idea es tomar prestados trozos de otros servicios o aplicaciones y cruzarlos con el fin de producir un servicio novedoso y unificado. Los *mashups* rinden honor a la vieja frase: «el todo es más que la suma de las partes». La creación de un *mashup* sería algo así como cocinar: simplemente hay que coger los diferentes ingredientes (servicios) que están a nuestra disposición y combinarlos de forma apropiada para crear un plato delicioso (un *mashup*).

La verdad es que las empresas están empezando a darse cuenta de la importancia de los *mashup*, y su máximo exponente es Google con su Google Maps (la aplicación que más *mashups* tiene). La idea es simple: Google ha puesto a disposición de la comunidad de Internet un servicio de mapas online (Google Maps) y lo ha abierto a través de APIs para que cualquiera pueda desarrollar aplicaciones sobre él. Lo ha hecho sabiendo que millones de desarrolladores de todo el mundo son capaces de idear servicios que ni siquiera la gente de Google podía imaginar.

Pero no solo Google apuesta por los *mashups*; hay muchas más empresas de Internet que apuestan por esta filosofía, como eBay, Microsoft con Windows Live, Yahoo!, YouTube, Flickr o Amazon.com, por poner algunos ejemplos. Existen *mashups* de múltiples servicios: de mapas (Google Maps), de fotos (Flickr), de compras (Amazon), de vídeos (YouTube)...



Esquema 1.

Normalmente, los **mashups** se basan en el uso de contenidos de terceros a través de fuentes o interfaces públicas. Entre las fuentes más utilizadas para crear *mashups* están Google Maps, Amazon, Flickr, YouTube, Ebay y Yahoo, por ejemplo. La facilidad con la que estas plataformas permiten integrar sus contenidos ha propiciado un gran éxito de las aplicaciones web híbridas.

Los tipos de fuentes de información más habituales que se utilizan en una aplicación web híbrida son:

- » Información proveniente de servicios web, disponible mediante diversos protocolos, e información estructurada utilizando formatos de intercambio como **JSON** o **XML**.
- » En ocasiones, el proveedor del servicio ofrece también una interfaz de programación (**API**) para facilitar el acceso a los datos. Es el caso de las **API** de compañías como Google, Yahoo!, Flickr, Microsoft o Amazon.
- » En otras ocasiones, los datos se ofrecen de forma pública, utilizando protocolos de redifusión web (también conocida como sindicación web, que es una manera sencilla de poner ciertos contenidos de un sitio web a disposición de otros), como **RSS** o **Atom**, y puede ser necesario procesarlos para extraer la información necesaria.

Sindicación web: http://es.wikipedia.org/wiki/Redifusi%C3%B3n_web



- » Información generada y gestionada por el propietario de la aplicación web híbrida, como pueden ser datos internos de una empresa.

De forma menos habitual, podemos encontrarnos aplicaciones web que utilicen técnicas de ingeniería inversa para extraer los datos que se muestran en algunos sitios web, como puede ser el caso de los precios de los productos en las tiendas web. Estas técnicas se conocen por su nombre en inglés: *web scraping*. Actualmente, muchas empresas utilizan técnicas de *web scraping* para ver los precios de la competencia.

Por ejemplo, se podría montar una aplicación web híbrida que utilice la API de Google Maps e información de ubicación geográfica de las franquicias de una empresa para mostrar la localización de las tiendas en un mapa.

1.2. Arquitectura de los *mashups*

Todos los *mashups* siguen una arquitectura única, compuesta de tres partes:

- » **La fuente de donde provienen los datos.** Son aquellos sitios de los cuales se recoge la información para crear la nueva página web. Normalmente viene a través de API o librerías, las cuales hacen más sencillo el acceso a los servicios ofrecidos.

Para algunas webs, como las de noticias, es más habitual encontrar que los datos se obtienen a través de otros protocolos web como RSS, REST y Web Service.

Sin embargo, no todos los sitios están adaptados para entrar a formar parte de un *mashup*. En estos casos, se hace lo que en inglés se llama *screen scraping*. Esta técnica consiste en extraer, mediante una herramienta especializada, información de los contenidos de las páginas proveedoras, traduciéndolas a un lenguaje que se pueda utilizar. La dificultad de esta técnica viene dada por la limitada capacidad de la traducción de un contenido que está enfocado en primera instancia a los usuarios y que, por ende, se encuentra en lenguaje natural.

» **El sitio que almacena el *mashup*.** Es donde el *mashup* se alberga, y en ocasiones, también donde se ejecuta. Los *mashups* pueden ejecutarse tanto en la parte del servidor como en la parte del cliente.

- *Mashups* ejecutados en servidor.
- *Mashups* ejecutados en cliente.
- *Mashups* ejecutados en servidor y cliente.

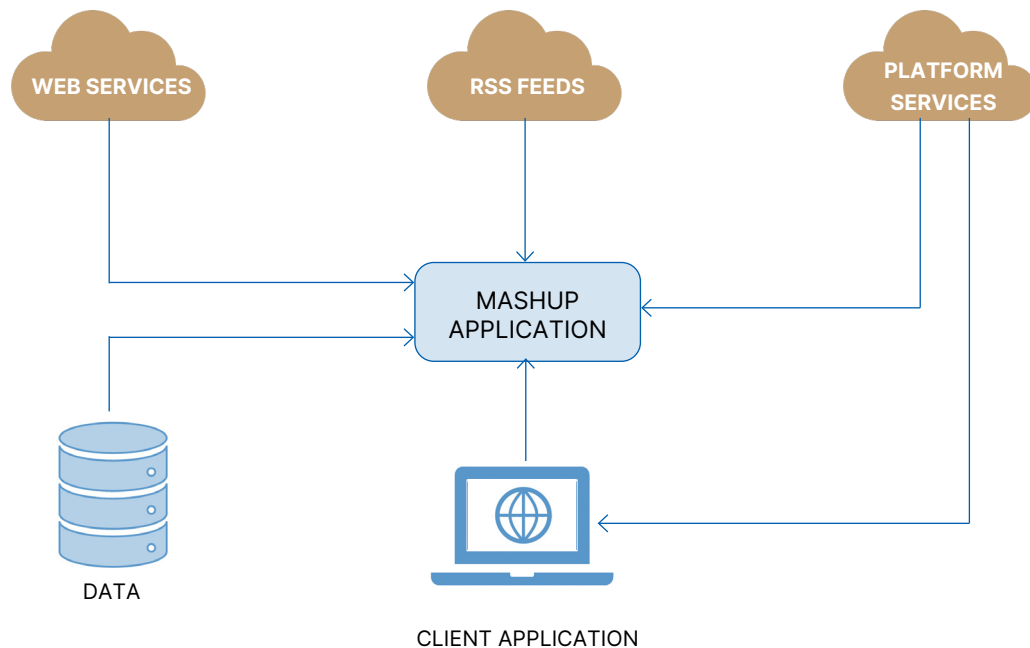
Este sitio se hará eco de los datos obtenidos de las fuentes del primer punto y, por lo tanto, no es dueño de ellos. Aquí se debe diferenciar la parte lógica del *mashup* de su parte física, puesto que no tienen por qué localizarse en el mismo sitio. Cuando hablamos del lugar que aloja el *mashup*, nos referimos al emplazamiento donde la parte lógica del *mashup* reside, que no tiene por qué ser donde se ejecute.

Esta última aserción viene a distinguir dos situaciones que pueden ocurrir, dependiendo de la implementación del *mashup*. Por un lado, se puede implementar como una web dinámica del lado del servidor, de forma que este sea el lugar donde se ejecuta el sitio. Por otro, se puede tener la opción de utilizar tecnologías como Ajax, actualmente muy extendida, que permite ejecutar la página en el lado del cliente.

La introducción de Ajax en este tipo de sitios entra dentro de lo que se llama «aplicaciones ricas de Internet» (RIA, rich Internet applications). Como se puede deducir, esta inclusión puede traer numerosos beneficios, más aún teniendo en cuenta el dinamismo que caracteriza su construcción. Algunas de estas ventajas son la reducción de la sobrecarga del servidor en el que se aloja el *mashup*, al recoger toda la información y después tener que enviársela al cliente, y una mejor interacción con el cliente, al no recargar la página entera.

Lo más frecuente, sin embargo, es encontrarse aplicaciones que utilizan ambas formas de ejecución, recogiendo parte de los datos en el lado del servidor, y otra parte en el lado del cliente, intentando optimizar al máximo la utilización de las técnicas de recuperación de datos, así como la experiencia del cliente.

»El **navegador de cliente**. Aquí es donde se renderiza gráficamente la aplicación y donde tiene lugar la interacción con el usuario. Como se ha comentado anteriormente, hay ocasiones en los *mashups* son ejecutados en el navegador del cliente.



Esquema 2.

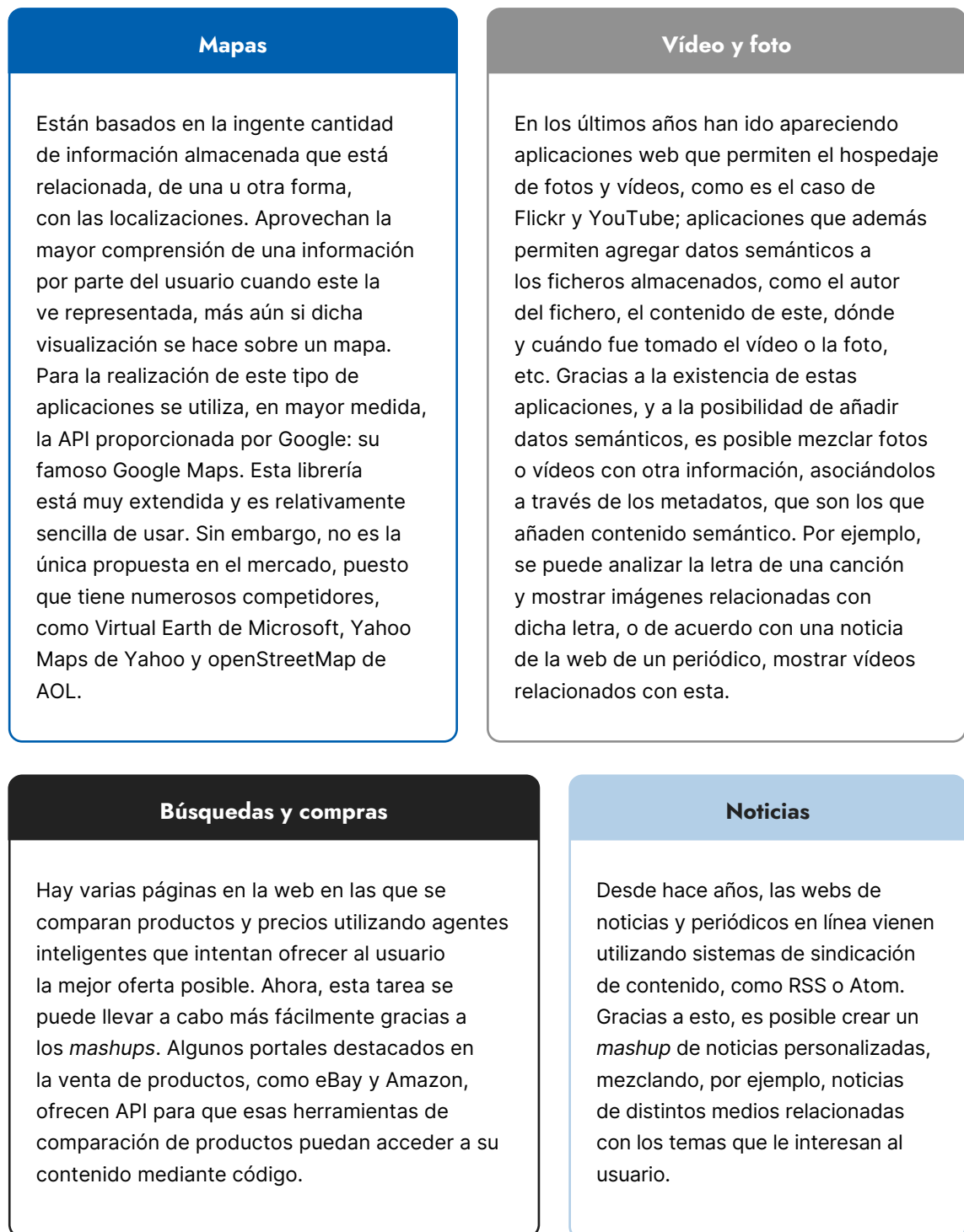
Los *mashups* deben diferenciarse de simples embebidos de datos de otro sitio para formar un documento compuesto. Un sitio que permite al usuario embeber vídeos de YouTube, por ejemplo, no es un sitio *mashup*. Como ya hemos comentado, el sitio mismo debe acceder a información externa a él usando una API y procesar esos datos de modo que incrementen su valor para el usuario.

1.3. Tipos de *mashups*

No hay unanimidad entre las diferentes clasificaciones de *mashups* que se pueden hacer. Existen diferentes clasificaciones de los *mashups*, entre las que destacan las siguientes:

A. Tipos de *mashups* atendiendo al contenido

Existen distintos tipos de *mashups*, con diferentes formas de presentar la información, y también con diferentes fines. A continuación, se describen los géneros más populares.



Esquema 3.

B. Tipos de *mashup* atendiendo a su localización

Generalmente, los *mashups* se almacenan y ejecutan en una misma localización. Esta puede variar dependiendo de los requisitos, y las alternativas que existen son: *mashups* ejecutados en servidor, *mashups* ejecutados en cliente o *mashups* ejecutados en cliente y servidor.

MASHUPS EJECUTADOS EN SERVIDOR

En este caso, el contenido se genera de forma dinámica en el servidor, usando tecnologías de generación de contenidos como Java, CGI, PHP o ASP. Una vez generado el contenido, este es enviado al navegador del cliente, el cual se encarga de mostrarlo.

VENTAJAS:	INCONVENIENTES:
» Menor consumo de recursos en la máquina de cliente. » Independencia del navegador del cliente. Al ejecutarse en el servidor, no requiere de una tecnología específica en la máquina del cliente.	» Dependencia de un servicio de terceros. Al estar almacenado en un servidor de terceros, pueden producirse problemas como indisponibilidad, errores (p. ej., <i>timeout</i>), etc.

MASHUPS EJECUTADOS EN CLIENTE

El contenido puede también ser generado de forma dinámica en la máquina del cliente, en el navegador a través de un script (como Javascript) o applets, o en una aplicación independiente. El cliente se encarga de solicitar la información inicial a las fuentes primarias y generar el contenido final.

VENTAJAS:	INCONVENIENTES:
Independencia de un servidor de terceros para la generación del <i>mashup</i>, excepto en el caso de las fuentes primarias. Generalmente, menor tiempo de generación del <i>mashup</i>, ya que tenemos una máquina dedicada al efecto.	Mayor volumen de información que hay que descargar. En lugar de tener que descargar el contenido final, es necesario descargar toda la información de las fuentes primarias.

MASHUPS EJECUTADOS EN SERVIDOR Y CLIENTE

En ocasiones, debido a ciertos requisitos, o dado el carácter heterogéneo de las fuentes primarias, puede producirse la necesidad de tener que generar un *mashup* «mixto», que se ejecute parte en la máquina de cliente, y parte en la máquina de servidor.

Tabla 1.

A modo de resumen, podemos determinar que los *mashups* ejecutados en servidor son independientes de la plataforma de cliente. Los ejecutados en el navegador de cliente tienen una cierta dependencia, ya que suele ser necesaria la instalación de *applets* (códigos Java que se ejecutan en el cliente; actualmente ya no se utilizan), extensiones, etc., en el navegador. Por último, los ejecutados en una aplicación independiente (*widget*) suelen ser los más dependientes de la plataforma, dado que habitualmente se trata de aplicaciones compiladas para una aplicación específica.

1.4. Ventajas e inconvenientes del uso de *mashups*

Es evidente que el uso de *mashups* proporciona una nueva forma de entender la visualización de los datos. Ahora es posible que un desarrollador, sin mayor esfuerzo que utilizando dos servicios accesibles a través de la red que han sido realizados por otros, cree un sitio web rico en información y que, en la mayoría de los casos, puede ser leído fácilmente en un primer vistazo. No es necesario disponer de grandes recursos, ni siquiera en el lado del servidor, pues todos los datos son recogidos de las fuentes a partir de las cuales se construye el *mashup*. Únicamente hay que aportar una amplia dosis de creatividad para que el producto final interese a los usuarios de la red.

La utilización de *mashups* se está viendo tan extendida precisamente por su facilidad de construcción y por lo atrayente que resulta que en una única página se fundan las ventajas de otras dos ya interesantes de por sí. Ni que decir tiene su amplio repertorio de posibles funcionalidades dentro de una empresa, al mezclarse con las herramientas más importantes ya extendidas, entre las cuales destaca la ayuda en la toma de decisiones al poder visualizar distintas situaciones, con el incentivo de estar toda la información visible sin necesidad de acudir a herramientas de *data warehouse*, sobre todo cuando el tamaño de la información no es eminentemente grande.

Desafortunadamente, no todo son ventajas; existe una dependencia insalvable con los proveedores de las fuentes de datos. Sin ellos, el *mashup* no existiría; si la información en ellos no es actualizada, el *mashup* tampoco se actualiza; si no ofrece datos verídicos, el *mashup* tampoco es fiable; si sus datos no son accesibles por una posible caída del servidor, el *mashup* tampoco puede mostrar la información pertinente, etc. Eso sin entrar a comentar la propiedad intelectual al utilizar software desarrollado por otras personas distintas al creador del *mashup*, o a la Ley de Protección de Datos. Por trabajar en la red, también se crean algunos riesgos, como que esta esté saturada y el *mashup* tarde en la recogida de datos y, por tanto, en mostrar al usuario su solución. Esto en un sitio web es un problema grave, puesto que una respuesta rápida es fundamental para que el usuario se quede en la página y vuelva a visitarla, que es, al fin y al cabo, para lo que se hacen los sitios web.

Otro aspecto que se debe tener en cuenta son las dificultades que se puede encontrar el desarrollador a la hora de implementar su *mashup*. Si bien antes se ha comentado que la creación de un *mashup* no supone ningún reto, hay que saber que no siempre es así. Uno de los grandes problemas que se pueden plantear es la integración de los datos cuando estos son muy heterogéneos entre sí.

Puedes encontrar más información comparativa entre aplicaciones nativas e híbridas en la sección **Descarga de documentos** de la unidad, en el archivo **Aplicaciones nativas frente a aplicaciones híbridas**.

02 Ventajas y mecanismos para la reutilización de código

2.1. ¿Reutilización de código?

Se suele acceder a las aplicaciones web híbridas, o **mashup**, a través de una interfaz pública o mediante **API**. Se supone que el concepto **mashup** implica integración fácil y rápida, ya que, si tuviésemos que programarlo todo en el mismo sistema, lenguaje de programación, la aplicación web que había que desarrollar sería técnicamente inviable en lo que respecta a tiempo y dinero; para ello, contamos con empresas como **YouTube**, **Microsoft**, **Google** o **Amazon** (entre otras), que nos proveen de una gran variedad de servicios para que los podamos integrar en nuestras aplicaciones. Este tipo de servicios serán ofrecidos y consumidos a través del protocolo de comunicaciones **HTTP**.

Hemos de tener en cuenta que incluir, por ejemplo, un vídeo de YouTube integrado en nuestra aplicación o proyecto no implica que sea una aplicación web híbrida. Para que sea una web híbrida, nuestra aplicación origen debe acceder a la información o procesar los datos que nos proporcione el servicio web, API, etcétera.

Una aplicación web híbrida o **mashup** implica que debe acceder a datos o procesar información a través del sitio que nos ofrece el proveedor, ya que este servicio de nuestro proveedor nos va a dar un valor añadido a nuestra aplicación, y sin esta ayuda sería técnicamente imposible ofrecer estos servicios.

2.2. Arquitectura de una aplicación web híbrida

La arquitectura de una aplicación web híbrida está compuesta por tres partes principales. Por un lado, lo más importante es el proveedor de contenidos —que ofrece servicios como mapas, geolocalizaciones, API para conectar con servicios de AdWords o transcripciones de audio a texto—, como el gran y todopoderoso **Google**. En el centro estaría situado el servidor de la nueva página híbrida, y, por último, el cliente, que es quien accede a la página.



Esquema 4.

Proveedor de contenidos

Es la fuente de datos. Los datos suelen estar disponibles a través de una interfaz pública o utilizando una API, con diferentes protocolos, como RRS, Atom, *screen scraping* o servicios web.

RSS

En otras ocasiones, los datos se ofrecen de forma pública utilizando protocolos de redifusión web (también conocida como sindicación web: es una manera sencilla de poner ciertos contenidos de un sitio web a disposición de otros); por ejemplo, RSS o Atom. Puede haber que procesarlos para extraer la información necesaria.

Redifusión web. http://es.wikipedia.org/wiki/Redifusi%C3%B3n_web

RSS (*really simple syndication*): tecnología basada en una estructura XML que permite compartir contenidos de una web determinada. Distribuye contenidos de una web sin necesitar un navegador.

Es una tecnología que envía automáticamente los titulares de un medio a un programa lector o agregador. Por ejemplo, elpais.com incorpora este servicio gratuito en su edición digital, para que sus lectores conozcan al instante las noticias más relevantes.

<https://servicios.elpais.com/rss/>

También puedes agregar, por ejemplo, los RSS de elmundo.es en tus servicios web favoritos.

<http://rss.elmundo.es/rss>

Las siglas RSS hacen referencia a la tecnología basada en fuentes web que hace posible que los proveedores de contenidos on-line publiquen nuevos contenidos en páginas web de forma estandarizada. Las fuentes web se pueden encontrar en portales de noticias, blogs, foros o wikis, aunque también en páginas web dinámicas, en las que es habitual que estén sujetas a actualizaciones. En cualquier caso, el objetivo de los RSS es informar a los usuarios de Internet de forma continua sobre nuevos contenidos a través de un lector de noticias. El flujo de información parte del destinatario, que, por interés, se abona al *web feed*, concebido como *pull media*. En estos casos, se suele utilizar un software de lector de *feeds* (por lo general, una *app* o *web app*). Los usuarios se abonan a canales para recibir noticias actuales de los proveedores de contenidos de manera automática. Los URL que enlazan a estos *feeds* pueden obtenerse a través del navegador web.

ATOM

Tecnología basada en XML que permite compartir contenidos de una web determinada. A diferencia de RSS, puede contener más información, y, al mismo tiempo, más compleja y consistente. Esto conlleva un mayor grado de dificultad asociada a esta tecnología.

Las principales diferencias de Atom frente a RSS:

1. RSS es un formato de fuente más popular y ampliamente utilizado, mientras que la fuente Atom todavía se usa como una alternativa a RSS.
2. El *feed* Atom es un estándar del IETF (Grupo de Trabajo de Ingeniería de Internet), mientras que el *feed* RSS no lo es.
3. Los *feeds* Atom proporcionan una variedad de cargas útiles, mientras que RSS proporciona texto sin formato o HTML escapado como cargas útiles.
4. Atom usa el atributo estándar `xml:lang` para especificar un idioma para elementos individuales contenidos en el *feed*, mientras que RSS no puede hacer eso.
5. Atom admite el uso de identificadores de recursos internacionales, mientras que RSS no lo admite.
6. Los *feeds* Atom tienen elementos `<summary>` y `<content>` separados, pero RSS no tiene elementos para contener un resumen. Solo puede contener descripción.



SCREEN SCRAPING

De forma menos habitual, podemos encontrarnos aplicaciones web que utilicen técnicas de ingeniería inversa para extraer los datos que se muestran en algunos sitios web, como puede ser el caso de los precios de los productos en las tiendas web. Estas técnicas se conocen por su nombre en inglés: *web scraping*.

Una finalidad muy utilizada en muchas empresas es obtener los precios de los productos de la competencia: accede a la página del proveedor a través de ingeniería inversa y procesa el contenido HTML recopilando la información deseada; extrae la información en forma de texto.

Web scraping, o raspado web, es una técnica utilizada mediante programas de software para extraer información de sitios web. Normalmente, estos programas simulan la navegación de un ser humano en Internet, ya sea utilizando el protocolo HTTP manualmente o incrustando un navegador en una aplicación.

Está en desuso, debido a los siguientes inconvenientes que presenta:

- »No todas las webs autorizan su uso.
- »Complejidad.
- »Sobrecarga de los servidores.
- »Propensión a fallos inesperados.

4

API

Se trata de un conjunto de definiciones y protocolos que se utilizan para desarrollar e integrar el software de las aplicaciones, de forma que se permite la comunicación entre dos aplicaciones de software a través de un conjunto de reglas. Presenta las siguientes ventajas:

- » Fiabilidad, escalabilidad, flexibilidad.
- » Sencillo y documentado.
- » El proveedor dispone de servidores específicos para este servicio, lo que le otorga robustez y una gestión de errores adecuada.

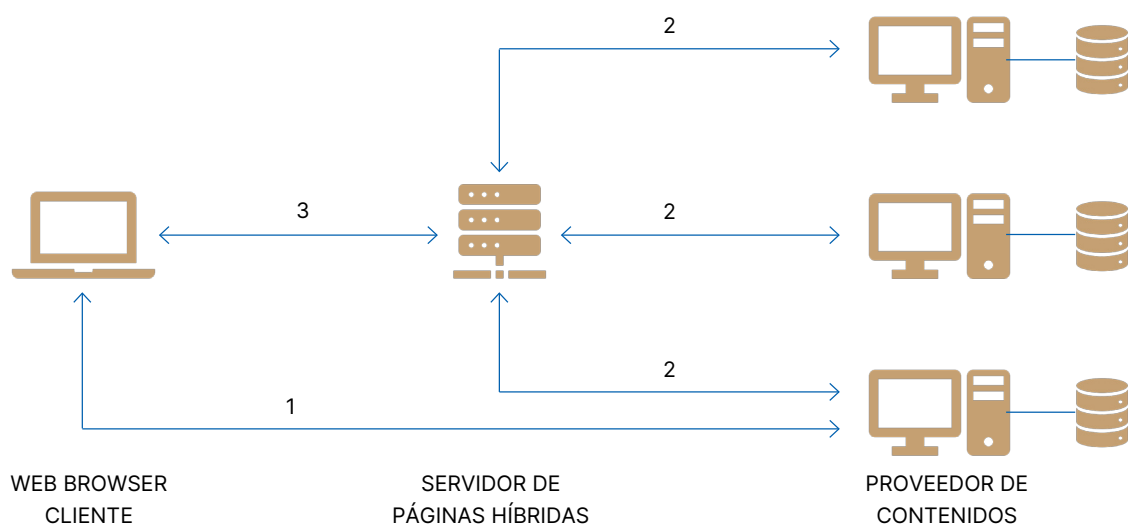
Ejemplos de API: <https://www.twilio.com/es-mx/>, para conectar con WhatsApp a través de API REST, API de Telegram a través de API REST y API Google Speech para la transcripción de audio a texto.

Sitio *mashup*

Es la aplicación web nueva que provee de un nuevo servicio utilizando diferentes fuentes de información de las que no es dueño. El sitio *mashup* hará uso de las tecnologías de las que disponga el proveedor de contenidos para conseguir generar la aplicación híbrida.

Cliente web

Es la interfaz del usuario del *mashup*. Es una aplicación web. Aquí el contenido también puede ser mezclado en el mismo navegador del cliente. Se pueden utilizar lenguajes como JavaScript y AJAX.

2.3. Comunicación en la arquitectura *mashup*

Esquema 5.

Vamos a ver el esquema de las diferentes vías de comunicación entre el navegador web del cliente y el proveedor de contenidos, pasando o no por el servidor de aplicaciones o páginas híbridas.

1. El navegador cliente accede directamente a la información del proveedor de contenidos mediante JavaScript.
2. El servidor de aplicaciones obtiene los datos directamente del proveedor de contenidos; estos se mezclan en el servidor y son enviados al navegador del cliente.
3. El navegador del cliente accede al servidor web y extrae el contenido almacenado.

2.4. División por categorías de los *mashups*

Suelen estar divididas en cuatro grandes categorías: mapas; fotos y vídeos; búsquedas y compras, y, por último, noticias.

A. *Mashups* de mapas

Muestran información sobre un mapa o imagen de satélite.

<http://www.estrellateyarde.org/mashups/ejemplo-de-mashup-con-google-maps-y-php>

Estos *mashups* están muy extendidos en multitud de aplicaciones web en las que es necesario precisar la situación de algo, como pueden ser los portales inmobiliarios.

https://www.idealista.com/	https://www.fotocasa.es/es/
https://www.pisos.com/	https://www.tucasa.com/

En todos estos portales está integrado Google Maps.

B. *Mashups* de fotos y vídeo

Estos *mashups* generan el contenido a partir de servidores web que almacenan grandes cantidades de fotografías y vídeos. El proveedor suele permitir realizar algunas tareas, como editar, ordenar o compartir, entre otras.

<https://www.flickr.com/services/api/>

Con esta API podemos ver cómo crear una web nueva híbrida que acceda a estos contenidos.

C. *Mashups* de compras

Permiten realizar búsquedas de las mejores ofertas comerciales.

<https://www.programmableweb.com/category/prices>

D. *Mashups* de noticias

Seleccionan fuentes de noticias a través de los protocolos **RSS** y **Atom**. Se tiende a leer las noticias desde el navegador del cliente, evitando visitas a los portales que proveen de las noticias.

2.5. Futuro de los *mashups*

Hemos visto que la mayoría de los *mashups* utilizan fuentes a través de protocolos RSS y Atom; es decir, obtenemos información mediante XML o JSON en sus comunicaciones.

Hoy en día, para cualquier desarrollo de aplicaciones software en el mundo empresarial haremos uso de API para obtener información, que podremos integrar en nuestro proyecto; los datos nos vendrán implícitamente en formato XML o JSON, y el desarrollador será capaz de gestionar estas fuentes de información en su aplicación.

Para dicha integración, lo que se está utilizando actualmente es tecnología **REST Web Services**.

Cabe mencionar que REST ha estado tomando fuerza a una velocidad impresionante y más con la llegada de **Node.js** y las bases de datos **NoSQL**, como **MongoDB**. Sin embargo, el hecho de que REST tome fuerza no significa que le esté quitando protagonismo a SOAP, pues recordemos que con la llegada del Internet de las cosas (IOT) cada vez se conectan más dispositivos a Internet que necesitan ser integrados (una gran oportunidad para REST), y es donde REST está tomando la delantera.

Por otra parte, se está viendo ya un cierto declive del RSS en lo que respecta a la publicación de noticias mediante ese sistema.

Los factores de declive de las RSS hay que buscarlos en el cambio de hábitos de los internautas. La mayoría de los usuarios se informa hoy día a través de las redes sociales, como Instagram, Twitter o Facebook, que ya transmiten las noticias en tiempo real.

Aparte de esto, el formato audiovisual gana peso frente al protagonismo del texto que representaban las RSS, y la abrumadora cantidad de información que recibe un usuario es ya suficiente como para que además quiera apuntarse a RSS.

La opción de «Suscríbete y recibe las últimas noticias en tu email» también ha quitado usuarios. Tuvo que ver, además, el cierre en 2013 de Google Reader, el lector de RSS más utilizado antaño, junto a la clausura de varios agregadores de contenidos: incluso el navegador Mozilla Firefox, que ofrecía lector RSS, lo eliminó también en 2018. Los grandes medios de comunicación han retirado de sus secciones de suscripciones directas de RSS.

Otro de los temas que se están afianzando frente a **SOA**, el antiguo competidor de **REST**, son los **Microservicios**.

Microservice Architecture

Es un estilo de desarrollo arquitectónico que permite crear aplicaciones como una colección de pequeños servicios autónomos desarrollados para un dominio comercial. Es una variante de la arquitectura de estilo estructural, que ayuda a organizar las aplicaciones como una colección de servicios débilmente acoplados. La arquitectura de microservicios contiene servicios detallados y protocolos ligeros. Actualmente, muchas empresas están adoptando esta arquitectura de **microservicios**.

Podemos tomar como ejemplo una aplicación de comercio electrónico desarrollada con arquitectura de **microservicio**. En esta arquitectura de microservicios, cada microservicio se centra en una sola capacidad comercial. *Búsqueda, Calificación & revisión y Pagos* tienen cada uno su instancia (servidor) y se comunican entre sí.

En la arquitectura monolítica, todos los componentes se unen en un solo módulo. Pero, en la arquitectura de microservicios, se distribuyen en módulos individuales (microservicios) que se comunican entre sí, como se muestra en el ejemplo de microservicios anterior.

<https://www.guru99.com/comparison-between-web-services.html>

<https://www.guru99.com/microservices-tutorial.html#1>

03 Repositorios

Al utilizar servicios de terceros para desarrollar una aplicación web híbrida, deberemos tener en cuenta que en ocasiones existen condiciones y límites al uso que puedes hacer de estos.

La mayoría de las grandes compañías que proveen servicios web al usuario, como Google, Amazon Web Services (AWS) o Azure de Microsoft, requieren un registro previo y ofrecen unas condiciones para su uso gratuito que dependen del servicio al que necesites acceder. Algunos de estos servicios ofrecen una versión adicional de pago con mejores condiciones. Por ejemplo, actualmente Google ofrece los siguientes límites de acceso gratuito para los siguientes servicios:

»Google Tasks5000 consultas diarias.

»Google Maps25 000 consultas diarias.

»Google Custom Search100 consultas diarias.

3.1. OAuth2

El protocolo estándar de autorización, OAuth2, permite a una aplicación externa obtener acceso a información de carácter privado de un tercero a través de un servicio web. Para ello establece un acuerdo de acceso a ella entre la aplicación externa, el servicio web y el propietario de los datos a los que se solicita el acceso.

Por ejemplo, si una aplicación «X» solicita a Google acceso a los calendarios del usuario **dwes**, Google pedirá a **dwes** permiso indicándole qué aplicación es la que solicita el acceso y a qué información. Si el usuario **dwes** otorga permiso, la aplicación «X» podrá acceder a los datos que solicitó a través del servicio de Google.

Para crear los identificadores necesarios para utilizar las API de Yahoo!, es necesario utilizar OAuth. Al utilizar una cuenta de Google para registrarnos como desarrolladores, Yahoo! necesita utilizar los servicios de Google para acceder a parte de nuestra información. Para

ello, Yahoo! remite una solicitud a Google, y este, a su vez, muestra una pantalla informando de que Yahoo! solicita información nuestra y nos pide permiso para darle acceso.

OAuth2 funciona de forma similar pero ligeramente distinta, dependiendo de quién solicite acceso a la información. En nuestro caso, supondremos que el solicitante será siempre una aplicación web.

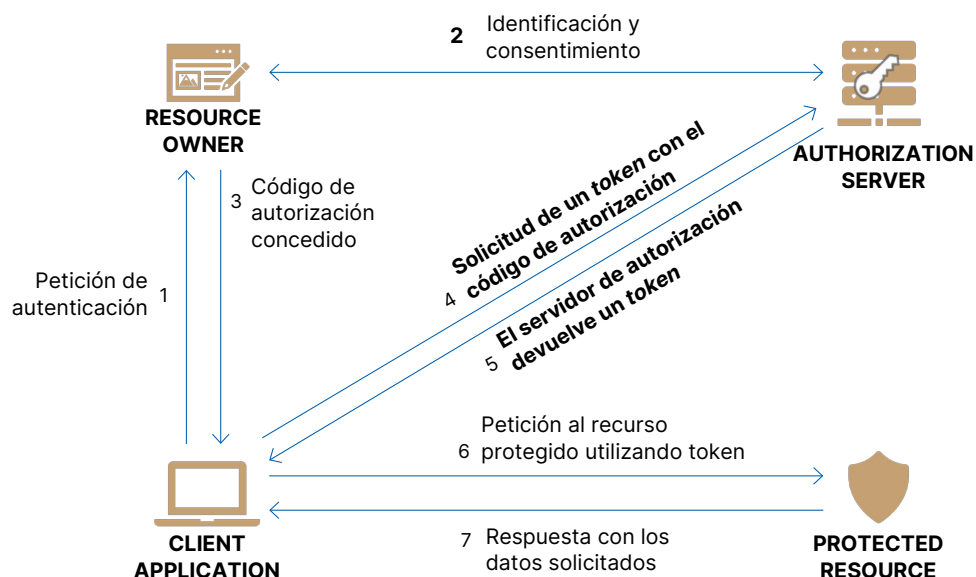
Veamos, por ejemplo, qué sucede cuando nuestra aplicación necesita acceder a información personal del usuario a través del servicio de Google Tasks. En este caso, los pasos que se seguirán son los siguientes:

La aplicación web se comunica con el servidor de autorización OAuth2, indicando la información a la que quiere acceder y el tipo de acceso a esta.

El servidor de autorización OAuth2 requiere al usuario de la aplicación web que inicie sesión con su cuenta de Google (si aún no lo ha hecho), y lo redirige a una página en la que le pide su consentimiento para otorgar acceso a su información privada.

Si el usuario da su consentimiento, el servidor de autorización OAuth2 devuelve a la aplicación web un código de autorización.

- » Antes de poder acceder a la información privada del usuario, la aplicación web debe intercambiar ese código o *token* de autorización por otro código o *token* de acceso. Es decir, desde la aplicación cliente se solicita un *token* de acceso con el código de autorización al servidor de autorización. Punto 4 y 5 de la imagen.
- » Utilizando este *token* de acceso, la aplicación puede utilizar el servicio de Google Tasks para gestionar la información privada del usuario, dentro de los límites de acceso que se han otorgado.
- » Los códigos de acceso tienen un tiempo de vida limitado. Cuando caducan, la aplicación ha de comunicarse de nuevo con el servidor de autorización OAuth2 para obtener un código de refresco.



Esquema 6.

3.2. JWT

La autenticación de *token* web JSON (**JWT**) se utiliza para verificar la propiedad de los datos JSON. JWT no es cifrado, sino que determina si se puede confiar en los datos porque se verifica su propiedad. JWT es un estándar abierto (RFC 7519) que permite que la información se transmita de forma segura entre dos partes como un objeto JSON. Los JWT se firman digitalmente utilizando un par de claves pública/privada o un secreto.

3.3. JSON y XML

Muchas de las operaciones que se llevan a cabo cuando utilizas un servicio web implican la obtención de cierta información por parte de este. La información obtenida puede ser bastante sencilla o tener cierto grado de complejidad. Por ejemplo, cuando utilizas un servicio de geocodificación para averiguar las coordenadas concretas de una dirección, obtienes básicamente esas coordenadas. Pero cuando utilizas un servicio como Google Directions para averiguar la ruta que se debe seguir entre dos puntos, la respuesta que obtienes es una ruta compuesta por una serie de indicaciones que hay que seguir para llegar al destino.

Los formatos más utilizados por los servicios web para dar formato a esas respuestas son dos: JSON y XML. En algunos casos, tendrás que adaptar tu código al tipo de respuesta que ofrezca el servicio.

Otros servicios permiten que escojas el tipo de respuesta que prefieras. Veamos cómo se pueden procesar de forma sencilla desde PHP mensajes en ambos formatos.

A partir de la versión 5.2 de PHP, se incluye por defecto la extensión JSON. Su funcionamiento es muy sencillo. Incorpora dos funciones para tratar con cadenas de texto en notación JSON:

json_decode. Decodifica una cadena de texto JSON y la transforma en un objeto PHP (opcionalmente, también se podría convertir en un array con la opción `true`).

```
<?php
$json = '{"a":1,"b":2,"c":3,"d":4,"e":5}';
var_dump(json_decode($json));
var_dump(json_decode($json, true));
?>
```

`json_encode`. Función inversa a la anterior. Devuelve una cadena de texto en notación JSON a partir del contenido de una variable PHP.

```
<?php
$arr = array('a' => 1, 'b' => 2, 'c' => 3, 'd' => 4, 'e' => 5);
echo json_encode($arr);
?>
```

Salida: {"a":1,"b":2,"c":3,"d":4,"e":5}

Esquema 7.

Así como las opciones para trabajar con JSON desde PHP están bien definidas, para utilizar XML hay más variedad de herramientas para escoger. La extensión **SimpleXML**, habilitada por defecto a partir de PHP 5.1.2, facilita la tarea de extraer información de un documento XML. Vamos a ver su funcionamiento. Extensión SimpleXML: <http://es.php.net/manual/es/book.simplexml.php>.

La extensión convierte un documento XML en un objeto de la clase **SimpleXMLElement**. Puedes cargar el documento:

» desde una cadena de texto, utilizando la función `simple_load_string`;

```
$xml = simplexml_load_string($cadena);
```

» desde un fichero, utilizando la función `simplexml_load_file`. Puedes utilizar una dirección URI como origen; por ejemplo:

```
$xml = simplexml_load_file('http://localhost/dwes/ut8/config.xml');
```

Los nodos y atributos del documento XML se convierten en atributos. Los nodos pasan a ser *arrays* que contienen a su vez como elementos los atributos y subnodos del documento XML. Por ejemplo, para acceder al resumen (*summary*) de una ruta en formato XML obtenida a partir del servicio Google Directions, podrías utilizar:

```
$ruta = simplexml_load_file('ruta al servicio');
$ruta->route[0]->summary
```

Ruta de Google Directions.

<https://developers.google.com/maps/documentation/directions>

Es importante que conozcas cómo procesar documentos XML para extraer información de estos. En el manual de PHP puedes encontrar información sobre la utilización de la extensión SimpleXML. Extensión SimpleXML: <http://www.php.net/manual/es/book.simplexml.php>.

3.4. Elaboración de documentos con formato RSS

Los documentos con formato RSS tienen una estructura simple, donde los elementos básicos son las etiquetas de XML `<rss>`, `<channel>` e `<item>`.

ETIQUETA	FUNCIÓN
<code>rss</code>	El elemento <code>rss</code> es el más importante de un <i>RSS feed</i> . Este define la versión de RSS utilizada y tiene que estar presente en cualquier documento RSS.
<code>channel</code>	El elemento <code>channel</code> describe el canal RSS y contiene metadatos importantes.
<code>item</code>	El elemento <code>item</code> sirve para destacar los bloques de los artículos.

Tabla 2.

Todos los documentos XML comienzan con una etiqueta que pone de manifiesto tanto la versión de XML como el código de caracteres empleado. En el ejemplo que nos ocupa, esto hace referencia a XML 1.0 y a UTF-8.

```
<? xml version='1.0' encoding='UTF-8' ?>
```

A la declaración inicial de XML le sigue la etiqueta `rss`, la cual indica la versión de RSS correspondiente y contiene, en el ejemplo actual, referencias a especificaciones adicionales que se pueden integrar en forma de espacios de nombres XML. En el ejemplo, esto se muestra a través de las distinciones semánticas conforme a Dublin Core (`dc`) y Atom (`atom`).

```
<rss xmlns:dc="http://purl.org/dc/elements/1.1/"  
xmlns:atom="http://www.w3.org/2005/Atom" version="2
```

Los contenidos propios del documento RSS están encofrados en la etiqueta `channel`. Esta alberga la descripción general del *news feed* (`title`, `description`) y los metadatos (`language`, `copyright`, `docs`, `generator`, `managingEditor`, `webMaster`), los logos y los enlaces.

```
<channel>  
<title>NYT > Home Page</title>  
<link>  
http://www.nytimes.com/pages/index.html?partner=rss&emc=rss  
</link>  
<atom:link rel="self" type="application/rss+xml" href="http://www.  
nytimes.com/services/xml/rss/nyt/HomePage.xml"/>  
<description/>
```

```

<language>en-us</language>
<copyright>Copyright 2017 The New York Times Company</copyright>
<lastBuildDate>Mon, 27 Mar 2017 08:19:34 GMT</lastBuildDate>
<image>
<title>NYT > Home Page</title>
<url>
https://static01.nyt.com/images/misc/NYT_logo_rss_250x40.png
</url>
<link>
http://www.nytimes.com/pages/index.html?partner=rss&emc=rss
</link>

```

Los elementos incluidos en la etiqueta **channel** se subdividen en elementos obligatorios y elementos opcionales.

ELEMENTOS OBLIGATORIOS	FUNCIÓN
Title	El elemento title contiene el título del canal RSS.
Link	El elemento link contiene el enlace a la página web.
Description	El elemento description contiene una descripción del canal RSS.
ETIQUETAS OPCIONALES	FUNCIÓN
language	El elemento language indica cuál es el idioma del canal RSS.
copyright	El elemento copyright contiene datos sobre los derechos de autor del canal.
atom:link	El elemento link adquirido de la especificación de Atom permite definir la relación existente con el elemento enlazado. El valor predeterminado es rel="alternate" , es decir, una representación alternativa del recurso actual. En este caso, el URL del feed de noticias.
docs	El elemento docs incluye el URL para la documentación del formato RSS empleado.

generator	El elemento generator incluye un <i>string</i> que muestra con qué programa se ha generado el canal.
managingEditor	El elemento managingEditor incluye la dirección de correo electrónico de la persona responsable de los contenidos del canal.
webMaster	El elemento webMaster integra el correo electrónico de la persona encargada de solucionar los problemas técnicos del canal. En el ejemplo actual, esto no tiene lugar en el sentido de la especificación.
image	El elemento image especifica un gráfico que se muestra junto con el canal. Las etiquetas <url> , <title> y <link> son obligatorias.

Tabla 3.

Además, la etiqueta **channel** contiene cada uno de los bloques **item**, que permiten mostrar la vista preliminar del artículo. Cada bloque **item** representa a un artículo, y en el ejemplo tomado en este apartado alberga las etiquetas **<title>**, **<link>**, **<description>**, **<category>**, **<dc:creator>**, **<pubDate>** y **<guid>**.

```
<item>
<title>
.....
</title>
<link>
.....
</link>
<guid isPermaLink="true">
.....
</guid>
<atom:link rel="standout" href="http://www.nytimes.com/2017/03/26/us/
politics/trump-republicans-tax-cuts.html?partner=rss&emc=rss"/>
<media:content url="https://static01.nyt.com/images/2017/03/27/
us/27TAXES-sub/27TAXES-sub-moth.jpg" medium="image" height="151"
width="151"/>
<media:description>
.....
</media:description>
<media:credit>.....</media:credit>
<description>
.....
```



```
<dc:creator>.....</dc:creator>
<pubDate>.....</pubDate>
<category domain="http://www.nytimes.com/namespaces/keywords/
des">.....</category>

</item>
```

04 Creación de aplicaciones web híbridas

En este apartado vamos a centrarnos en el desarrollo de aplicaciones híbridas y las distintas tecnologías que se pueden emplear para su desarrollo.

Este tipo de *apps* híbridas funcionarán independientemente del dispositivo en el que se ejecuten: móviles, tabletas, ordenadores, *smart-tv*...

Para el desarrollo de este tipo de *apps* se emplean lenguajes como **Javascript**, **ECMAScript**, **Typescript**, **HTML5**, **CSS**..., que se pueden implementar mediante *frameworks* como Ionic o Angular.

Veremos un completo análisis sobre las distintas tecnologías de desarrollo de aplicaciones híbridas.

4.1. Herramientas utilizadas para crear *mashups* o aplicaciones híbridas

Las herramientas generalmente más utilizadas para el desarrollo de aplicaciones web híbridas son Google Maps, Amazon Web Services, Del.icio.us, Flickr, Microsoft, Yahoo!, Twitter y YouTube. Un gran número de ellos tienen recursos que aportan al diseño e implementación de los *mashups*.

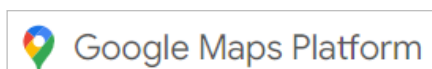
Es habitual que la integración de datos se obtenga desde API abiertas.

FUENTES MÁS UTILIZADAS PARA APLICACIONES HÍBRIDAS

Google Maps	Permite la incorporación de Google Maps en páginas web de desarrolladores externos, utilizando una interfaz JavaScript simple.
Amazon Web Services	Amazon API Gateway es un servicio completamente administrado que facilita a los desarrolladores la creación, la publicación, el mantenimiento, el monitoreo y la protección de API a cualquier escala.
Flickr	Flickr API permite obtener fotos e imágenes desde la galería de Flickr.
YouTube	Mediante la API de YouTube, se pueden cargar vídeos, administrar listas de reproducción y suscripciones, actualizar la configuración del canal y más cosas.
Ebay	La API eBay permite acceder a la base de datos de eBay a través de una licencia que se concede a los afiliados a tal efecto. Se trata de que los titulares de dicha licencia puedan proveer de contenido en tiempo real a sus respectivos sitios a través de los artículos puestos en venta en cualquiera de los sitios de eBay.
Yahoo!	Yahoo! API tiene un conector listo para conectar con muchos sistemas, como Gmail, PrestaShop, OpenCart, Facebook, WooCommerce de Wordpress, etcétera.

Tabla 4.

A. Google Maps



Con la API de Google Maps Platform puedes integrar mapas dinámicos, mostrar mapas estáticos dentro de tu aplicación e integrar instrucciones sobre cómo llegar orientadas a varios medios de transporte, además de información sobre el tráfico en tiempo real y calcular las distancias y los tiempos de viaje para varios orígenes y destinos; identifica rutas cercanas mediante coordenadas.

Puedes encontrar toda la información en <https://developers.google.com/maps?hl=es-419>.

B. Amazon Web Services



API Gateway gestiona todas las tareas implicadas en la aceptación y el procesamiento de hasta cientos de miles de llamadas a API simultáneas; entre ellas, la administración del tráfico, la compatibilidad con CORS (característica de seguridad del navegador, que restringe las solicitudes HTTP de origen cruzado que se inician desde secuencias de comandos que se ejecutan en el navegador), el control de autorizaciones y acceso, la limitación controlada, el monitoreo y la administración de versiones de API. API Gateway no requiere pagos mínimos ni costos iniciales. Se paga por las llamadas a las API que se reciben y por la cantidad de datos salientes transferidos; además, con el modelo de precios por niveles de API Gateway se pueden reducir los costes a medida que se cambie la escala de uso de las API.

Más información en <https://aws.amazon.com/es/api-gateway/>.

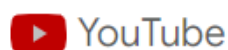
C. Flickr



Flickr posee una interfaz de programación de aplicaciones (API) abierta. Esto significa que cualquier persona puede crear su propio programa para presentar datos públicos de Flickr (como fotos, vídeos, etiquetas, perfiles o grupos) de maneras nuevas y diferentes. El **App Garden** es un lugar donde los desarrolladores pueden exhibir las aplicaciones que crearon y donde tú puedes encontrar nuevas formas de explorar Flickr.

Documentación sobre la API de Flickr en <https://www.flickr.com/services/api/>.

D. YouTube



Se puede usar la API para buscar vídeos que coincidan con términos de búsqueda específicos, temas, ubicaciones, fechas de publicación y mucho más. El método `search.list` de la API también admite búsquedas de listas de reproducción y canales.

Más información de la API de YouTube en <https://developers.google.com/youtube/v3>.

E. eBay



El uso de la API es relativamente sencillo, pero requiere que los desarrolladores lean atentamente toda la información disponible. Asimismo, pueden acceder a la gran comunidad de desarrolladores API.

API de afiliados disponible en <https://pages.ebay.es/afiliados/api.html>.

F. Yahoo!



API

Yahoo! es un popular portal de Internet que combina una serie de servicios. Su lista incluye Yahoo! Correo, noticias con subsecciones temáticas (política, finanzas, deportes, entretenimiento, estilo de vida, tecnología, etc.), así como tendencias, clima, calificaciones deportivas y otros contenidos. Otro componente clave del servicio es Yahoo! Búsqueda. Yahoo! ofrece a los usuarios configurar la integración de sus servicios con sitios, programas, aplicaciones móviles y otro software de terceros a través de una API disponible en <https://developer.yahoo.com/api/>. Incluye el protocolo OAuth de autorización y autenticación que le permite alojar, reenviar y acceder a datos seguros.

4.2. Librerías PHP de acceso a servicios web para la creación de aplicaciones híbridas

IMPORTANTE

Para crear aplicaciones web híbridas, necesitarás que tu aplicación web acceda a diversos servicios para obtener información. En muchas ocasiones, esos servicios estarán accesibles mediante HTTP. En otras ocasiones, especialmente cuando accedas a información privada de un usuario, necesitarás utilizar un protocolo seguro, como HTTPS.

Tanto si vas a generar tu propio código para acceder a un servicio web como si utilizas una API ya programada, es posible que necesites utilizar la librería **cURL**. Puedes encontrar más información en <http://es.php.net/manual/es/book.curl.php>. **cURL** es una librería (y también una herramienta para utilizar desde la línea de comandos) que permite utilizar de forma sencilla varios protocolos de comunicaciones para transmitir información. Admite,

entre otros, los protocolos HTTP, HTTPS, FTP, TFTP, Telnet, LDAP, SMTP, POP3 e IMAP. Es importante que te asegures de que tu instalación de PHP incluye esta librería. cURL es una librería de terceros que simula solicitudes y respuestas HTTP de una manera mucho más eficiente. Puede manejar solicitudes HTTP asíncronas y comunicaciones complejas, como funciones de devolución de llamada o transferencia continua de punto de interrupción. También es adecuada para realizar solicitudes FTP basadas en dominios cruzados. Además, se puede usar en diferentes aplicaciones, como la configuración de *proxy* y *web scrapping*, etcétera.

```
<?php
```

```
// Para obtener contenidos de una web desde una URL
$url = "https://www.geeksforgeeks.org/";
// Inicializar una sesión CURL.
$ch = curl_init();
curl_setopt($ch, CURLOPT_RETURNTRANSFER, 1);
// Grab URL and pass it to the variable
curl_setopt($ch, CURLOPT_URL, $url);
$result = curl_exec($ch);
// Devuelve los contenidos de la página.
echo $result;
```

Otra de las funciones que se pueden utilizar para acceder a un servicio web o recuperar contenidos de una web es la función `file_get_contents()`: esta función de PHP se utiliza para recuperar el contenido de un archivo. El contenido se puede almacenar como una variable de cadena. Como alternativa, también simula transacciones HTTP, involucrando solicitudes a través del método GET y respuestas usando el método POST. Principalmente, es más adecuado para manipulaciones HTTP simples y para obtener respuestas JSON de una sola línea.

```
<?php
```

```
// Leyendo contenidos desde la
// página web GeeksforGeeks
$homepage = file_get_contents(
    "https://www.geeksforgeeks.org/");
echo $homepage;
```

FILE_GET_CONTENTS	CURL
Maneja comunicaciones HTTP simples.	Maneja comunicaciones HTTP complejas.
Admite operaciones HTTP GET y HTTP POST simples.	Admite certificados HTTP PUT, además de solicitudes GET y POST.
No admite almacenamiento en caché, cookies, etc.	Admite almacenamiento en caché, informes de progreso de cookies, etc.
Utiliza los protocolos HTTP y HTTPS para las comunicaciones.	Utiliza los protocolos HTTP, HTTPS, FTP, FTPS.
Se puede utilizar para leer el contenido del archivo	Se puede utilizar para leer, editar, actualizar y eliminar archivos del servidor.
Lento en funcionamiento	Funcionamiento seguro y rápido
Fácil de entender	Complejo de entender

Tabla 5.

**VÍDEOLAB**

Para consultar el siguiente vídeo, escanea el código QR o [pulsa aquí](#).

UAX FP